# Smart City Interop Platform

Plateforme SOC (Smart Operations Center) mêlant services REST, SOAP, GraphQL et gRPC sous une gateway Kong, orchestrés par SmartFlow et consommés par un tableau de bord React. Docker Compose fournit l'intégralité de l'environnement (bases PostgreSQL, Redis, services métiers, orchestrateur, gateway et client web).

## Monorepo layout

| Path | Description |
|---|---|
| `services/mobility-rest` | Spring Boot REST (réservations multimodales persistées Postgres). |
| `services/airquality-soap` | Service SOAP JAX-WS exposant AQI & polluants du Grand Tunis. |
| `services/emergency-grpc` | Spring Boot + gRPC + Postgres (alertes + flux temps réel + simulateur 5 s). |
| `services/directory-graphql` | Annuaire & tickets GraphQL (Spring GraphQL + Flyway). |
| `orchestrator/smartflow` | Façade Node/Express consommée par le dashboard (trip planning, métriques, alertes SSE). |
| `client/web-dashboard` | SPA React (Vite, TanStack Query, Playwright E2E). |
| `gateway/kong` | Routage + auth (JWT HS256 + key-auth, TLS optionnel). |
| `contracts/` | Source de vérité OpenAPI/WSDL/SDL/proto. |
| `docs/` | Historique projet par phase + références sécurité. |

## Quick start

```
# Prérequis : Docker Desktop en cours d'exécution
docker compose build
docker compose up -d

# Reconstruction ciblée (ex : frontend)
docker compose up -d --build web-dashboard
```

- Kong : `http://localhost:8000` (ou `https://localhost:8443` avec `gateway/certs`).
- Dashboard : `http://localhost:4173`.
- SmartFlow direct (dev) : `http://localhost:7000`.

Attendre les logs `Started <Service>` de chaque microservice avant d'appeler la gateway pour éviter les `502` initiaux.

## Authentication via Kong

Tous les appels proxifiés exigent deux en-têtes :

```
Authorization: Bearer <jwt>
x-api-key: citizen-api-key
```

- Consumers : `citizen-app` (`supersecretcitizen`) et `operator-app` (`supersecretoperator`).
- Génération HS256 documentée dans `docs/phase4/gateway-security.md`.

## Persistence & runtime ports

| Service | Port | Data volume |
|---|---|---|
| Mobility REST | 8080 | `pg_mobility_data` (`bookings`). |
| AirQuality SOAP | 8081 | In-memory dataset. |
| Emergency gRPC | 50051 (HTTP mgmt 9090) | `pg_emergency_data` (`emergency_alert`). Simulateur injecte une alerte synthétique toutes les 5 s. |
| Directory GraphQL | 8082 | `pg_directory_data` (annuaire/tickets). |
| SmartFlow | 7000 | Redis cache (`redis-cache`). |
| Kong | 8000 / 8443 | Pas de stockage (DB-less). |
| Web dashboard | 4173 | Build Vite. |

## Development & tests

```
# SmartFlow orchestrator
(cd orchestrator/smartflow && npm install && npm test)

# Services Spring Boot
(cd services/mobility-rest && mvn test)
(cd services/emergency-grpc && mvn test)
(cd services/directory-graphql && mvn test)

# SOAP build
(cd services/airquality-soap && mvn package)

# Frontend build + e2e
(cd client/web-dashboard && npm install && npm run build)
RUN_E2E=1 PLAYWRIGHT_BASE_URL=http://localhost:4173 \
  (cd client/web-dashboard && npx playwright install && npm run test:e2e)
```

Redis indisponible ? SmartFlow bascule automatiquement sur un cache mémoire TTL identique.

## Documentation & runbooks

- `Description.md` — synthèse exhaustive (architecture, services, endpoints, flux, tests, runbook).
- `FullReadme.md` — ancienne source longue, conservée pour l'historique.
- `docs/phase*/` — décisions par phase (vision, architecture, sécurité, client, conteneurisation, etc.).
- `docs/phase4/gateway-security.md` — politiques JWT + key-auth + TLS.

## Tips

- `docker compose logs -f <service>` pour vérifier l'état individuel.
- `docker compose up -d --build emergency-service` relance uniquement le service gRPC (incluant le simulateur d'alertes toutes les 5 s).
- Les appels GraphQL passent via `/api/directory`; pour tester en direct : `curl http://localhost:8082/graphql -d '{"query":"{ services { name } }"}'`.

Besoin d'un panorama complet (endpoints, payloads, flux réseau, procédures de test) ? Consultez `Description.md` fraîchement généré.